

2-dars: Class imbalanced problem, Oversampling va Undersampling

Class imbalanced problem nima (anyidea?)
Hozirgi vaqtda **real-world** datasetl u-n tabiiy muammo.

Example: Email soxta (5%) yoki soxta emas (95%) / transaction da firibgarlik bor (5%) yoki yo'g (95%)

Agar model shunday imbalanced datasetda train qilinsa nima bo'ladi?

• Natijada:

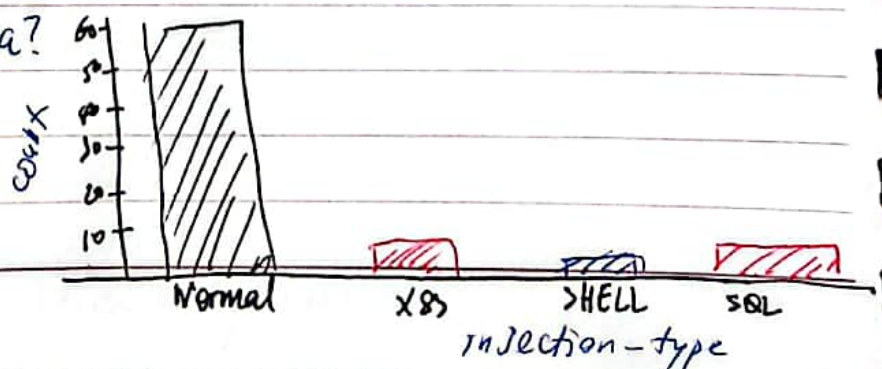
- Kichik miqdordagi classlarni ignore qiladi.
- Kichik miqdordagi class ustida past natija chigadi.
- Umumiy natija baland bo'lishi mumkin lekin **biased** performance bo'ladi va bu **overfittingga** sabab bo'ladi.

Oversampling va Undersampling $\Rightarrow$ Class imbalancedda yechim.

Sampling texnikasi:

- $\hookrightarrow$ Oversampling
- $\hookrightarrow$ Undersampling

Class imbalance nima?



- Sampling : Oversampling

Oversampling - 0 yoki 1 ichidagi qiymatlar samplelar hisoblanadi.

o Kichik miqdordagi classlardagi qiymatlarni oshirish va katta miqdordagi class soniga yettirish.

Oversampling techniques :

I. Random Oversampling $\Rightarrow$ Randomly duplicate qiladi (kam class/ni)

Misalan class 1 $\Rightarrow$ 90 va class 0 $\Rightarrow$ 10 ta shu class 0 ni class 1 ga tenglashtirib oladi.

II. SMOTE (synthetic minority Oversampling Technique)

III. ADASYN (Adaptive Synthetic Sampling)

Oversampling kichik miqdordagi class/ni qiymat/ni duplicate qiladi. yoki yangi sintetik data point yaratadi.

$$\rightarrow \text{New} = A + \lambda (B - A) \quad \lambda = [0:1] \Rightarrow 0 < \lambda < 1$$

Misalan: $A = [10, 20]$, $B = [20, 40]$ $\lambda = 0.5$

$$\text{New} = [10, 20] + 0.5([20, 40] - [10, 20])$$

$$N = [15, 30] \Rightarrow \text{Demak } [15, 30] \text{ - yangi synthetic sample}$$

ADASYN (Adaptive Synthetic Sampling)

↳ Modelga qiyin bo'layotgan minority sampellarini atrofida ko'proq data yaratadi.

Yani $\Rightarrow$ "Qaysi minority nuqtalar atrofida majority juda ko'p? Demak, shu joy model uchun qiyin. Shu yerga ko'proq **synthetic** data yaratishim k." "

Xo'jayotiy analog: Sizda 2 guruh bor javoni soatga talaba.

■ 10 ta yaxshi tayyorlanmagan talaba

■ 90 ta yaxshi tayyorlangan talaba

Modelga bu ikki guruhni ajratishni o'rgatqchisiz. Ammo ■ guruh juda kam.

ADASYN nima qiladi?

Avval ■ guruhdagi har bir odamga qaraydi va $u =$ atrofidagi odam / himligini qaraydi

Masalan:

Ali ■

■ ■ ■
■

} Ali atrofida asosan ■ minority class bor.

Demak, Ali joylashgan hudud oson.

Vali:

■ ■

■ Vali

■ ■

■

} Vali atrofida ko'pchilik ■

→ Demak Vali joylashgan hudud **qiyin**.
Model Vali kabi na'munalarni ajratishda qiyinlashadi.

ADASYN shunda nima deydi?

Ali →  → ozgina yangi data

Vali →  →  → ko'prog yangi data.

Yani ADASYN hamma joyga bir xil miqdorda data qo'shmaydi.

Qiyin joy → ko'prog data

Oson joy → kamrog data.

Formulasi:

$$r_i = \frac{\Delta_i}{k}$$

⇒ 1. Qiyinlik darajasi

$$g_i = \frac{r_i}{\sum r_i} G$$

⇒ 2. Qancha synthetic data yaratishni hisoblash.

Faraz qilaylik, **ADASYN** jami to'la yangi synthetic sample yaratish k-k. $G = 10$

Bizda 2ta minority sample bor:

-  Ali

-  Vali

1. Har biri = qiyinlik darajasi; Har biri = 5-ta. eng yag'in qo'shnisi bor.

Ali: ^{5ta qo'shmasi}  $\Rightarrow$ Engat 1 tasi: *minority*

Shu uchun:

$$r_{Ali} = \frac{1}{5} = 0,2$$

majority (above 1), *minority* (below 1)

Vali: ^{4 tasi majority} 

Shuning uchun:

$$r_{Vali} = \frac{4}{5} = 0,8$$

Demak:

Ali $\rightarrow$ o'son $\rightarrow 0,2$

Vali $\rightarrow$ qiyin $\rightarrow 0,8$

2. Endi synthetic data'ni taqsimlaymiz.

Formula:
$$g_i = \frac{r_i}{\sum r_i} \cdot G$$

Avval
$$\sum r_i = 0,2 + 0,8 = 1$$

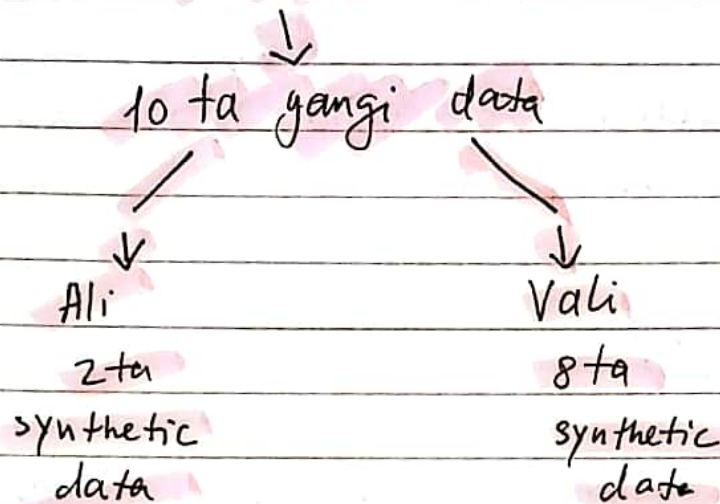
Ali u-n:
$$g_{Ali} = \frac{0,2}{1} \times 10 = 2 \Rightarrow$$
 Demak, Ali atro-

fida 2ta synthetic sample yaratamiz.

Vali uchun:
$$g_{Vali} = \frac{0,8}{1} \times 10 = 8 \Rightarrow$$
 Demak

Vali u-n 8ta sample yaratamiz

ADASYN



Coding xolotda

! pip install imbalanced-learn

```

import numpy as np
import pandas as pd
from collections import Counter # sana Bertsh u-n.
from imblearn.over_sampling import RandomOverSampler,
SMOTE, ADASYN

```

```

X = np.array([[1], [2], [3], [4], [5], [6]])
y = np.array([0, 0, 0, 0, 1, 1])

```

```

print("Original class distribution:", Counter(y))
↳ ... Counter({np.int64(0): 4, np.int64(1): 2})

```

```

ros = RandomOverSampler(random_state = 42)
X_ros, y_ros = ros.fit_resample(X, y)

```



```
print("\n Random Oversampling class distributions",
      Counter(y_ros))
print("Random Over sampling samples : \n", X
      X_ros.flatten())
```

SMOTE

```
smote = SMOTE (random_state = 42, n_neighbors = 1)
```

katta database
shart emu
olish

```
X_smote, y_smote = smote.fit_resample(x, y)
```

```
print("\n SMOTE class distribution:", Counter(y_smote))
print("SMOTE samples : \n", X_smote.flatten())
```

ADASYN

```
adasyn = ADASYN (random_state = 42, n_neighbors = 1)
X_ada, y_ada = adasyn.fit_resample(x, y)
```

```
print("\n ADASYN class distribution:", Counter(y_ada))
print("ADASYN samples : \n", X_ada.flatten())
```

flatten() → multidimensional array → 1D array

Undersampling:

↳ Majority classdagi qiymat/ni kamaytirish.

Asosiy texnikalari:

→ Random Undersampling

→ Cluster Centroids

→ Near Miss Algorithms

x	y
1	0
2	0
3	0
4	1
5	1

majority
minority

x	y
1	0
2	0
4	1
5	1

majorityda
o'chiramil

Centroid nima?

Tasavvur qil: Bir xonada 5ta odam turibdi. U/le v'ztasida bitta joy bor.

Manashu joy - centroid

Data'da ham huddi shunday:

* ← centroid = Barcha nuqtalar = o'rtacha joylashgan nuqtasi.

$$A = (2, 2), B = (4, 4), C = (6, 3)$$

$$C_x = \frac{2+4+6}{3} = 4 \quad C_y = \frac{2+4+3}{3} = 3$$

$$\text{Centroid} = (4, 3)$$

Centroid = barcha nuqta / = o'rtacha nuqtasi.

$$\text{Centroid} = \left(\frac{\sum x_i}{n}, \frac{\sum y_i}{n} \right)$$

Agar data 3D bo'lsa, z ham qo'shiladi:

$$\left(\frac{\sum x_i}{n}, \frac{\sum y_i}{n}, \frac{\sum z_i}{n} \right)$$

K-means - datalarni o'xshashligiga qarab k ta guruhga (cluster) ajratadigan algoritim

Masalan, $k = 3$ desak, 3 ta cluster topadi

Jarayon:

1. k ta centroid tanlaydi.
2. Har bir data pointni eng yaqin centroidga birlashtiradi.
3. Har bir cluster uchun yangi centroidni (o'rtacha nuqtani) hisoblaydi.
4. Centroidlar o'zgar maguncha 2-3 qadamni takrorlaydi.

Data

k ta centroid

Point $\rightarrow$ eng yaqin centroid

Yangi centroid

Takrorlash

Final Clusters

K-means $\Rightarrow$ eng yaqin markazga birlashtir $\rightarrow$ markazni qayta hisobla $\rightarrow$ takrorla.

Near Miss Algorithm $\Rightarrow$ Majority classni kamaytiradi (undersampling) va minority classga yaqin namunalarni saqlaydi.

U majority class ichida minority classga eng yaqin namunalarni saqlashga harakat qiladi.

Majority = 100

Minority = 20

$$d(x, y) = \sqrt{\sum_{j=1}^n (x_j - y_j)^2}$$

$$A = (2, 3), B = (5, 7)$$

$$d(A, B) = \sqrt{(2-5)^2 + (3-7)^2} = \sqrt{25} = 5$$

← Minority

← yagon majority → SHARLANADI

← uzog majority → OLIG TASHLANISHI mumkin.

Coding: Amaliyotda

```
import numpy as np
import pandas as pd
from collections import Counter
from imblearn.under-sampling import
    RandomUnderSampler, ClusterCentroids, NearMiss
```

```
X = np.array([1, 1], [2, 1], [3, 1], [4, 1], [5, 1], [6, 1])
y = np.array([0, 0, 0, 0, 1, 1])
```

```
print("Original class distribution: ", Counter(y))
```

```
# Random Undersampling
```

```
rus = RandomUnderSampler(random_state=42)
X_rus, y_rus = rus.fit_resample(X, y)
```

```
print("\n Random Undersampling class distribution: ",
      Counter(y_rus))
```

```
print("Random Undersampling samples: \n", X_rus)
```



```
# Cluster centroids
```

```
cc = ClusterCentroids (random_state=42)
```

```
X_cc, y_cc = cc.fit_resample(X, y)
```

```
print("\n Cluster Centroids class distribution:",  
      Counter(y_cc))
```

```
print(" Cluster Centroids samples:\n", X_cc)
```

```
# NEAREST MISS ALGORITHM
```

```
nm = NearMiss (version=1, n_neighbors=1)
```

```
X_nm, y_nm = nm.fit_resample(X, y)
```

```
print("\n Near Miss class distribution:", Counter(y_nm))
```

```
print(" Near Miss samples:\n", X_nm)
```

```
df_original = pd.DataFrame({'X1': X[:,0], 'X2': X[:,1],  
                           'y': y})
```

```
df_res = pd.DataFrame({'X1': X_res[:,0], 'X2': X_res[:,1], 'y': y_res})
```

```
df_cc = pd.DataFrame({'X1': X_cc[:,0], 'X2': X_cc[:,1], 'y': y_cc})
```

```
df_nm = pd.DataFrame(.....)
```

```
print("\n Original dataset:\n", df_original)
```

```
print("\n Cluster Centroids dataset:\n", df_cc)
```

```
→ df_res  
→ df_nm
```


x_1 x_2 class
 2 3
 3 4
 5 6
 2.6 3.6 0

gacha bitta o'yaratish kerak! $\alpha = (0, 1)$
 $x_{new} = x_{final} + \alpha (x_{before final} - x_{final}) \Rightarrow$
 $\Rightarrow x_{new} = (2, 3) + 0,6 ((2, 4) - (2, 3)) = (2, 3) + 0,6 (1, 1) = (2, 6)$

OVERSAMPLING (ADVANTAGES)

+

- Datani originalini saqlab qoladi.
- Minority classlarni ulushini oshiradi $\rightarrow$ minority classning value larini predict qilish; imkoniyatini paydo qiladi.
- Yetarli computing resourceslar bilan yaqinlashi ishlaydi.

-

- Overfittingga sabab bo'ladi (Random Oversampling)
- Training time oshib ketadi.

UNDERSAMPLING (ADVANTAGES)

+

- Training tezroq bo'ladi
- Sodda va effective

-

- Kuchim bo'lgan datalarni yongolishiga sabab bo'ladi
- Underfittingga sabab bo'lishi mumkin.

"OFFLINE TESTING"